

# OpenViking 开源 上下文数据库范式探索

演讲人：盛茂家

公司 / 职位：火山引擎 Viking 团队技术负责人

**AiCon**  
全球人工智能开发与应用大会

# 目录

01

## OpenViking 项目背景介绍

- 项目背景：全新定义的上下文数据库
- 快速开始：用最快的速度部署和使用 OpenViking

02

## OpenViking 设计和演进过程

- 依赖、部署、选型考虑
- 如何以 AI Agents 为中心进行多租户设计
- 记忆系统和知识检索系统的迭代

03

## OpenViking 应用场景和评估

- 多智能体协作、记忆能力、知识管理、工程代码检索
- 性能、成本和效率

04

## OpenViking 的愿景和展望

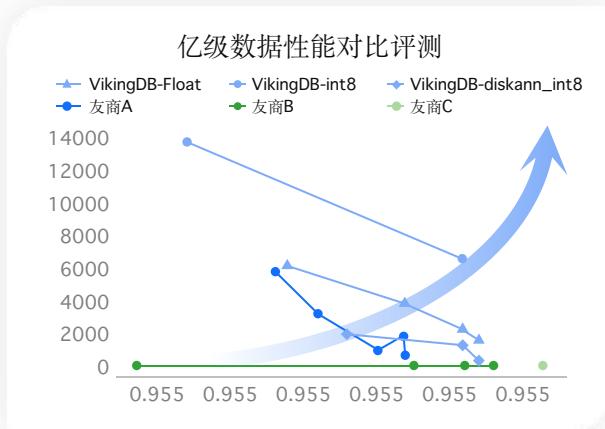
- 服务于 AI 原生团队
- 参与构建安全可靠的智能体软件栈

# 01 OpenViking 项目背景介绍

<https://openviking.ai/>

<https://github.com/volcengine/OpenViking>

最开始的问题：为什么给客户和业务最好的向量数据库、向量化模型，还是做不好上下文工程？



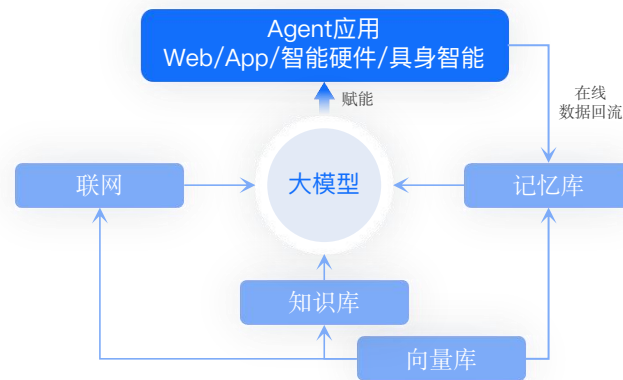
## 向量化模型和向量数据库

主要解决相关性度量和索引的性能问题，  
帮我们能在海量非结构化数据中召回可能  
相关的信息



## 在我们支持的业务和客户中

更多要回答的是：要将什么信息转换成检索对象，以及如何为 **Agent** 构建可检索的上下文

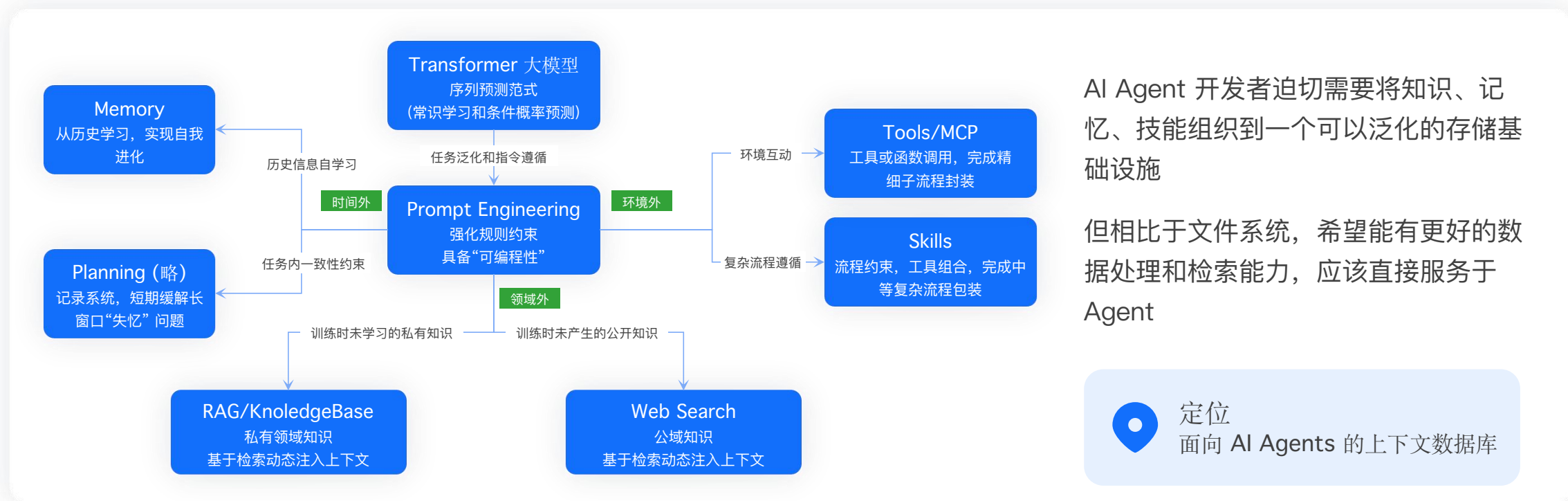


## Viking将上下文工程进行系统化设计

在我们的产品中，所有的数据都要有地方可以存放，因此对于 **Agent** 而言，除了本地 **Workspace** 之外的核心是知识和记忆

注: Viking 团队 19 年起持续支持抖音、TikTok、飞书等业务的非结构化检索, 火山引擎对外商业销售的产品包括: VikingDB 向量库、火山方舟知识库、Viking 记忆库

# 项目背景：全新定义的上下文数据库



而在 25 年思考如何解决相关问题时, 我们观察并认可两个 Agent 行业趋势:

1. 随着 Coding Agent 的能力上限持续突破, 文件系统正在成为 Agent 默认掌握的数据管理能力, Skills 等协议的出现正在强化这一能力
2. 在 Agent Workspace 也就是工作环境中, CLI 正在成为 Agent 可以拿来就用的最直接工具, 协议对接成本最低, 远好于 MCP/API/SDK

# 快速开始：用最快速度部署和使用 OpenViking

OpenViking 努力让你可以在 Linux/MacOS/Windows (X86/Arm) 以及云环境上快速部署和集成



(建议在 uv 环境下)  
uv pip install openviking

命令行安装



```
docker run -d \
  --name openviking \
  -p 1933:1933 \
  -v ~/.openviking:/app/.openviking \
  --restart unless-stopped \
  ghcr.io/volcengine/openviking:latest
```

我们已经将数据库服务、控制台、VikingBot 组装到一个容器，可以快速体验和交付

Docker 部署



<https://console.volcengine.com/vikingdb/openviking>

一键开通云上最佳实践

火山引擎购买



npm i -g @openviking/cli

开始体验：  
ov status  
ov ls

直接连接使用 CLI

为了保证开源版本极低的上手成本，OpenViking 默认使用单机形态，但是高可用和高性能推荐采购云上的分布式向量数据库与文件系统，一般建议：

个人使用：单机、私有化、组合 Agent Plan 或 Coding Plan  
生产使用：分布式、云上、组合企业级的模型推理方案



# 快速开始：用最快速度部署和使用 OpenViking

OpenViking 一经推出，就获得了社区的高度认同，成为全球智能体知识和记忆相关领域增长最快的开源项目（之一？）

Claude Code  
自动召回和自动捕获

Codex  
Hooks + 内置 MCP

OpenClaw  
Context engine 插件

OpenCode  
统一记忆工具插件

**Hermes**  
内置 OpenViking provider

MCP  
Trae / Cursor / Manus / ChatGPT

SDK  
Python / LangChain / LangGraph

☐ 自部署 OpenViking Server ☒ OpenViking Service

1 将 Hermes 接入 OpenViking Service

> API Key 获取：登录[火山引擎控制台](#)，开通 OpenViking Service 后，在对应服务的 API Key 管理 / API Keys 页面创建并复制。

> 终端

复制命令

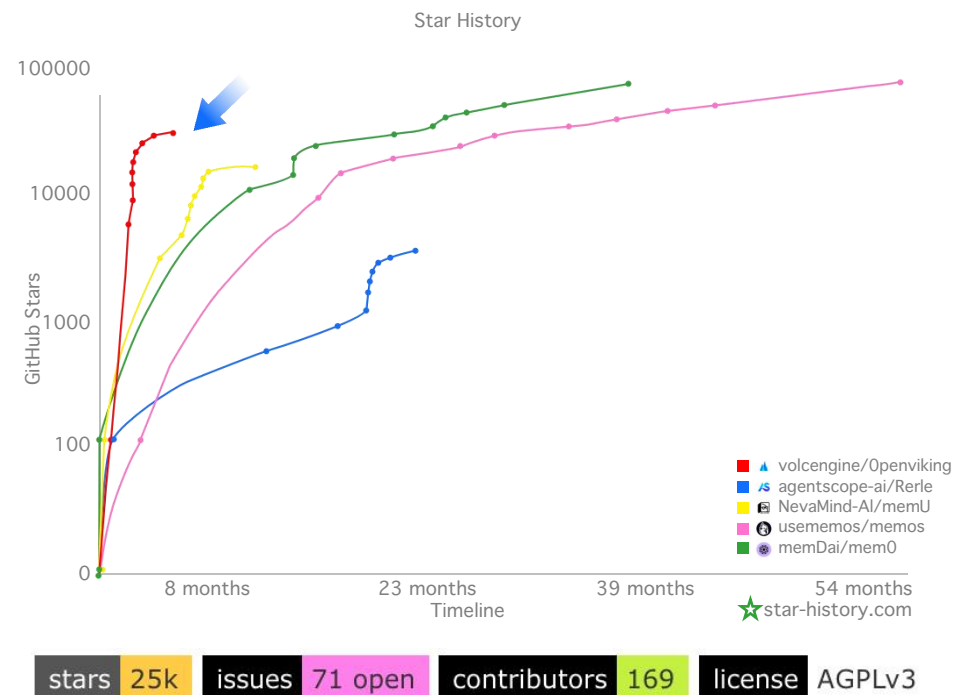
```
hermes memory setup
# 选择 openviking, 然后输入 endpoint / API Key
hermes config set memory.provider openviking
mkdir -p ~/.hermes
cat >> ~/.hermes/.env <<'EOF'
OPENVIKING_ENDPOINT=https://api.vikingdb.cn-beijing.volces.com/openviking
OPENVIKING_API_KEY=<your-api-key>
EOF
hermes memory status
```

2 重启并试用

> 终端

复制命令

```
hermes
# 输入：使用 viking_search 回忆 OpenViking Service 接入步骤。
```



我们正和整个开源社区合作，用一套统一规范接收大量智能体的接入，插件、CLI/Skill、MCP、SDK 等不同接入方式服务于不同场景  
多家顶尖国内大厂和研究机构也和我们建立了合作关系，探索在认知科学、企业智能化、AI4S 研究等领域的应用

# 02 OpenViking 设计和演进过程

<https://openviking.ai/>

<https://github.com/volcengine/OpenViking>



# ■ 依赖、部署、选型考虑

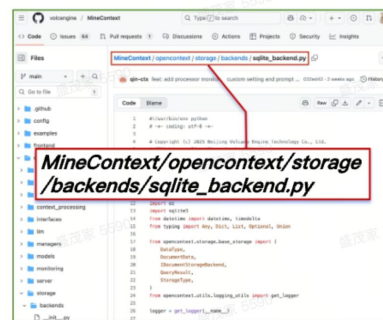
OpenViking 将最重的赌注压在了文件系统语义，但索引方面，我们结合 VikingDB 向量库进行了大量研究，最终其实整合了向量索引、文件系统索引、表/正倒排索引、图和链接结构

- 语义相关性: Vector Index > Graph > File System > Table
- 规模适应性: Vector Index ~ Table > Graph > File System
- 智能体适应性: Vector Index ~ File System > Table ~ Graph
- 自动化建模难度: Vector Index > Graph ~ File System > Table
- 模态通用性: Vector Index > File System > Table > Graph
- 索引查询效率: Vector Index ~ Table > Graph > File System
- 查询筛选能力: Table > File System > Graph > Vector Index
- 维度扩展能力: Table > Vector Index > Graph > File System

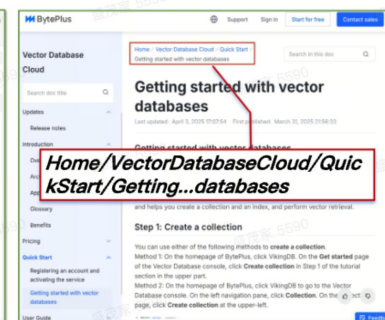
# ■ 依赖、部署、选型考虑

OpenViking 将最重的赌注压在了文件系统语义，但索引方面，我们结合 VikingDB 向量库进行了大量研究，最终其实整合了向量索引、文件系统索引、表/正倒排索引、图和链接结构

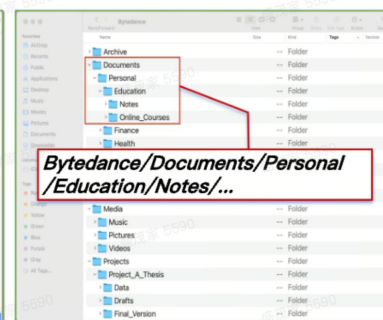
- 语义相关性: Vector Index > Graph > File System > Table
- 规模适应性: Vector Index ~ Table > Graph > File System
- 智能体适应性: Vector Index ~ File System > Table ~ Graph
- 自动化建模难度: Vector Index > Graph ~ File System > Table
- 模态通用性: Vector Index > File System > Table > Graph
- 索引查询效率: Vector Index ~ Table > Graph > File System
- 查询筛选能力: Table > File System > Graph > Vector Index
- 维度扩展能力: Table > Vector Index > Graph > File System



(a) Code Repositories



(b) Corporate Wikis



(c) File Systems

(d) Skills <https://code.claude.com/docs/zh-CN/skills>

(e) ARA 协议: <https://arxiv.org/abs/2604.24658>

文件系统是 Agent 访问和存储数据正确率最高的方式，核心在于目录化的组织结构，本来也是 Wiki、操作系统、代码仓库 等非结构化信息的最主流结构但是纯粹靠文件系统无法实现高效语义索引和多模态索引，因此我们需要结合向量数据库来重新设计

# ■ 依赖、部署、选型考虑

协议设计: CLI 优先

模仿 unix 文件系统命令, 让 LLM 快速理解

- 增 `ov add-resource https://example.com/guide.md`
- 删 `ov rm -r viking://resources/old-project/`
- 查 `abstract, overview, read, ls, tree, find, grep, glob, stat`  
`ov ls viking://resources/`  
`ov find "how to handle API rate limits"`
- 改 `ov write viking://resources/docs/api.md --content "# Updated\n\nNew content."`
- 存 采用文件系统和向量数据库作为底层存储依赖, 存储的内容保证 Agent 可直接读 (可直接作为 VLM 输入)

# 依赖、部署、选型考虑

| 产品特性        | OpenViking<br>上下文数据库 | VikingDB<br>向量库           | LocalFS/AGFS/TOS<br>文件系统 |
|-------------|----------------------|---------------------------|--------------------------|
| 数据操作        | 增删查改                 | 增删查改                      | 增删改（需要依赖其他应用实现查询）        |
| 协议设计        | HTTP / API<br>SDK    | HTTP / API<br>CLI、SDK     | HTTP / API<br>CLI、SDK    |
| 输入格式        | 文件，文本内容，链接，<br>对话历史  | 向量，标量<br>易于转为向量的特定模式文件或内容 | 文件                       |
| 语义检索        | 是（基于向量）              | 是（基于向量）                   | 否                        |
| 关键词检索       | 是（基于稀疏向量或Grep）       | 是（基于稀疏向量和关键词索引）           | 是（基于Grep）                |
| 保留层次化结构     | 是                    | 否                         | 是                        |
| 数据隔离        | 是（账号、用户、智能体）         | 是（基于标量）                   | 部分（基于user / group）       |
| Agentic遍历阅读 | 是                    | 否                         | 是                        |
| 支持数据自动摘要    | 是                    | 否                         | 否                        |
| 支持自动解析      | 是                    | 否                         | 否                        |
| 内置辅助智能体     | 是（bot）               | 否                         | 否                        |
| 支持数据修改追踪    | 是（计划内）               | 否                         | 否                        |
| 支持本地部署      | 是                    | 否                         | 部分                       |
| 支持云上托管      | 是（计划内）               | 是                         | 部分                       |
| 支持原生记忆插件    | 是                    | 否                         | 否                        |
| 支持原生RAG     | 是（bot）               | 否                         | 否                        |

## 协议设计：CLI 优先

模仿 unix 文件系统命令，让 LLM 快速理解

- 增 `ov add-resource https://example.com/guide.md`
- 删 `ov rm -r viking://resources/old-project/`
- 查 `abstract, overview, read, ls, tree, find, grep, glob, stat`  
`ov ls viking://resources/`  
`ov find "how to handle API rate limits"`
- 改 `ov write viking://resources/docs/api.md --content "# Updated\n\nNew content."`
- 存 采用文件系统和向量数据库作为底层存储依赖，存储的内容保证 Agent 可直接读（可直接作为 VLM 输入）

# ■ 依赖、部署、选型考虑

OpenViking 需要高性能的语义检索，在 VikingDB 中是如何实现的？

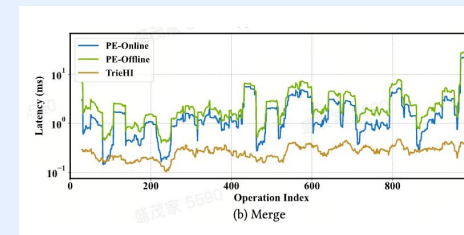
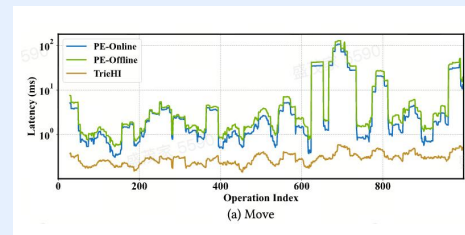
以 `ov find "docker deployment" -- uri "viking://resources/volcengine/OpenViking"` 为例

## 传统（向量）数据库为何低效

- 传统数据库 uri 只能通过 prefix 匹配，大多数数据库在实现时，无法进行有效的前缀索引，而逐行过滤性能极低
- 如果目录深度优先、数据规模不大，还不太明显，但如果目录规模达到十万-千万量级，将压垮整个数据库

## OpenViking 的做法

- 为目录匹配设计专用实时索引，既能满足过滤检索需求，还能实时调整以支持 mv 等批量操作，不再担心深层目录性能问题，例如
- 搜索用户记忆: `ov find "xxx" --uri "viking://users"`



# ■ 如何以 AI Agents 为中心进行多租户设计

OpenViking 需要处理不同的应用场景，在这些应用场景中，对“租户”“用户”“智能体”的定义是不太一样的

## SaaS Agent 应用

客服、公开角色

- 开发更复杂的 Agent 应用并对外使用，用户是海量不特定群体，每个用户的信息要隔离，但是不要进行单独的鉴权，因此引入 `peer_id` 的概念

## 企业 Agent 应用

虚拟员工、智能流程

- 企业具备更复杂的团队分层结构，因此需要划分不同的信息空间，我们需要引入类似于“组”的概念，在 OpenViking 里叫 `Account` 子账户

## 团队 Agent 应用

工作助手、效率工具

- 团队具备共享信息（知识资源、技能等），需要让一部分上下文共享，但保留用户对话和历史信息的隐私性，需要引入 `user_id` 和 `api_key` 进行权限隔离

## 个人 Agent 应用

助理、陪伴、创作

- 用户“养龙虾”、“AI Coding”等场景 Agent 记忆几乎等价于用户使用 Agent 时的记忆，用户上传的知识即 Agent 可以访问的知识



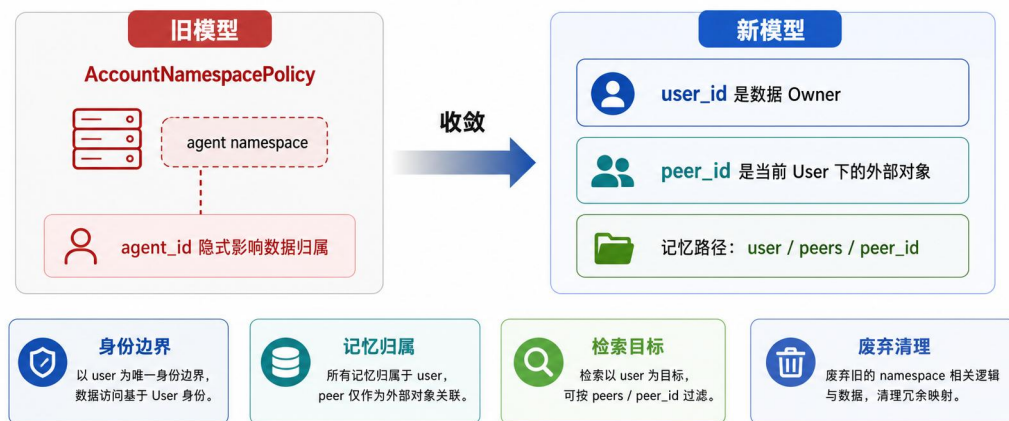
# 如何以 AI Agents 为中心进行多租户设计

OpenViking 0.4 开始支持全新的鉴权和数据隔离模型，在鉴权问题上不再区分 user/agent

- 租户空间划分：以 Account 为账号组进行划分，Account 之间完全隔离
- 多用户鉴权：以 user\_id 为数据的唯一持有人
- 多对象隔离：支持以 peer\_id 进行业务隔离，以避免不同说话人之间的信息干扰

## PR #2236

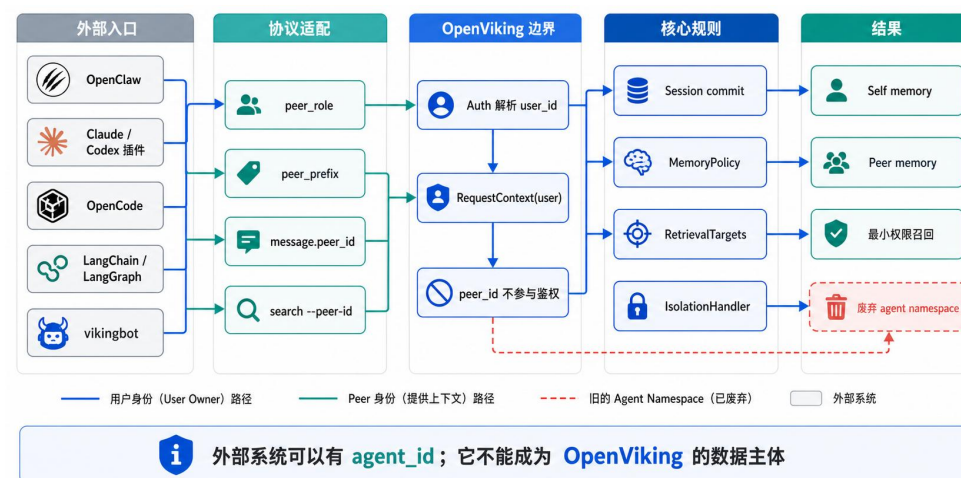
### 从 Agent Namespace 到 User /Peer 记忆模型



445 文件 | +7244 / -12197 | Breaking Change

## 端到端流程

### 外部 Agent 只提供 Peer，OpenViking 只认 User Owner



# 记忆系统和知识检索系统的迭代

OpenViking 将上下文分为三种主要类型：记忆、资源、能力。

## 记忆

主观

持续迭代

随时间变化和遗忘

主动加工

## 资源

客观

跟随外部变化

可以被引用

不做额外的加工

## 能力

主观或客观

可能需要持续进化

关于实践的知识

按需加工

# 记忆系统和知识检索系统的迭代

OpenViking 采用主题化记忆组织方式：任何需要记住的信息，都应该放在“合适”的类目下，然后持续迭代



# 记忆系统和知识检索系统的迭代

OpenViking 采用主题化记忆组织方式：任何需要记住的信息，都应该放在“合适”的类目下，然后持续迭代



## Event Schema

```
EventType: {{ Name of Your Event Memory }}
Description: {{ Detailed description of the memory you want to extract }}
Properties: [
  {
    // attribute that constitutes this type of event
    PropertyName: {{ Attribute name }}
    PropertyType: {{ Attribute Type }}
    Description: {{ A detailed description of the attribute }}
  },
  { // property two },
  { ... }
]
```

## Operators

LLM\_MERGE, SUM, MAX, AVG, TIME\_COMPRESS, COUNT

## Entity Schema

```
EntityType: {{ Name of Your Entity Memory }}
Description: {{ Detailed description of this entity memory }}
Properties: [
  {
    // attribute that constitutes this type of entity
    PropertyName: {{ Attribute name }}
    PropertyType: {{ Attribute Type }}
    Description: {{ A detailed description of the attribute }}
    AggregateExpression: {
      Op: {{ Aggregation operators }}
      EventName: {{ The event type name participating in the aggregation. }}
      PropertyName: {{ The name of the attribute in the event type that participates in the aggregation. }}
      IsPrimaryKey: {{True or False}}
    }
  },
  { // property two },
  { ... }
]
```

大模型抽取记忆的过程类似于：

消费对话和日志数据，然后在维护日志的仓库下持续迭代（打 patch）

## Algorithm 1: EUA: Patch-based Entity Update w/o LLM

**Input:** Entity schema  $S$ ; old entity  $E^{old}$ ; field-wise patches  $\{p_f\}$   
**Output:** Updated entity  $E^{new}$

```
1  $E^{new} \leftarrow E^{old}$ ;
2 foreach field  $f$  in  $S$  do
3    $(s, r) \leftarrow \text{PARSEPATCH}(p_f)$ ; //  $s=\text{SEARCH}$ ,  $r=\text{REPLACE}$ 
4   if  $s = \emptyset$  then
5     continue
6    $(i, j) \leftarrow \text{BESTAPPROXSPAN}(E^{old}[f], s)$ ; // min edit distance
7    $E^{new}[f] \leftarrow E^{old}[f]_{[0:i]} \parallel r \parallel E^{old}[f]_{[j:]}$ ;
8 return  $E^{new}$ ;
```

## VikingMem: A Memory Base Management System for Stateful LLM-based Applications

|                                                                               |                                                                            |                                                                                |                                                                |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------|
| Jiajie Fu <sup>†</sup><br>Zhejiang University<br>China<br>jiajiefu@zju.edu.cn | Junwen Chen <sup>†</sup><br>ByteDance<br>China<br>chenjunwen@bytedance.com | Mengzhao Wang <sup>*</sup><br>Zhejiang University<br>China<br>wmzwy@zju.edu.cn | Aoxiang He<br>ByteDance<br>China<br>heaoxiang@bytedance.com    |
| Maojia Sheng<br>ByteDance<br>China<br>shengmaojia@bytedance.com               | Xiangyu Ke<br>Zhejiang University<br>China<br>xiangyu.ke@zju.edu.cn        | Yifan Zhu<br>Zhejiang University<br>China<br>xtf_z@zju.edu.cn                  | Yunjun Gao<br>Zhejiang University<br>China<br>gaoyj@zju.edu.cn |

VLDB26

<https://arxiv.org/pdf/2605.29640>

## Event Instance

```
event_id: 25111513141
event_content: {
  tool: "Drawing Helper",
  description: "Used for drawing characters, landscapes, etc.",
  is_success: True,
  use_situation: user requested a character portrait with an emphasis on artistic sense."
  token_usage: 128,
  time_cost_sec: 35,
  feedback: "The user thinks that the drawing meets the requirements, especially noting that the eyes were drawn with great detail."
  thoughts: "execute successfully, meets requirements."
}
event_time: "2025-11-15 13:14:35"
```

Evolve

## Entity Instance

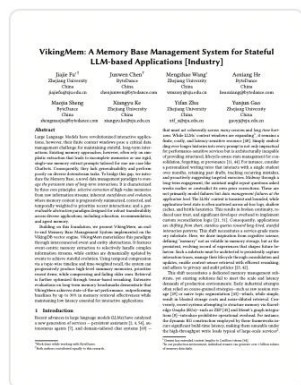
```
entity_id: 2509102214
entity_content: {
  tool_name: "Drawing Helper",
  description: "Used for drawing characters, landscapes, etc.",
  tool_call_times: 148,
  success_rate: 0.96,
  avg_token_usage: 149,
  avg_time_cost_sec: 48,
  suitable_for: "Drawing portraits and landscapes, especially when it comes to rendering and refining details."
  failure_cases: "The generated image exceeds 5 MB, thus the outcomes is abnormal."
  suggestions: "Invoke this when rendering detailed graphics"
}
last_updated: "2025-11-15 13:14:35"
```

# 记忆系统和知识检索系统的迭代

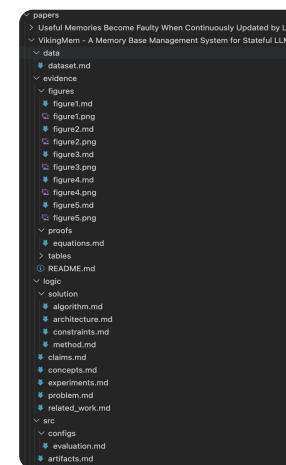
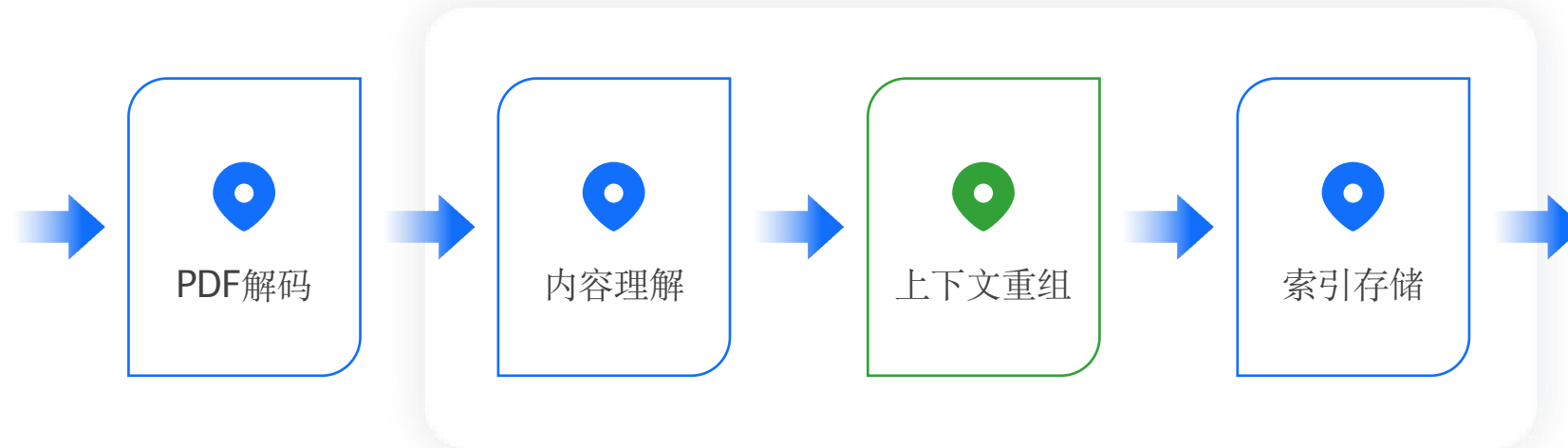
OpenViking 相对于传统 RAG: 为 AI Agents 重新组织数据, 所有入库数据可以直接作为 LLM/VLM 的输入



思考: OpenViking 为什么是数据库而不是文件系统?



VikingMem-  
VLDB26-Final.pdf

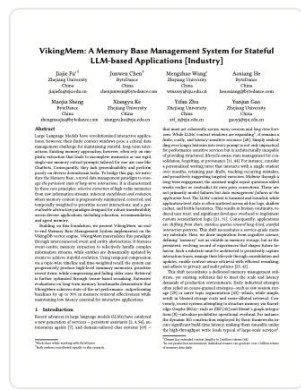


# 记忆系统和知识检索系统的迭代

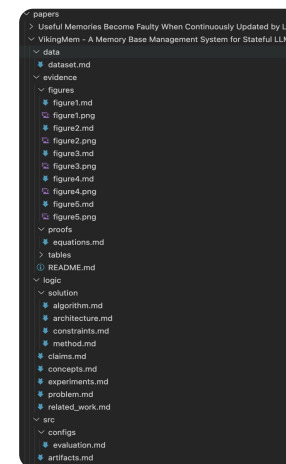
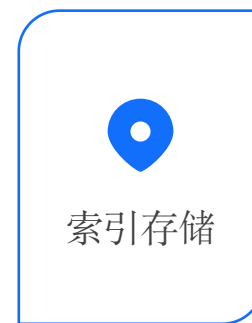
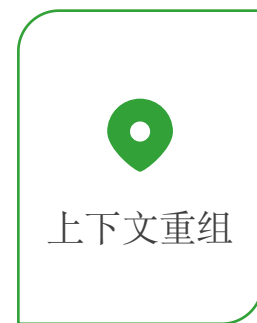
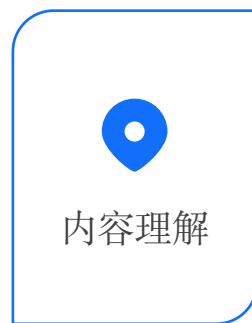
OpenViking 相对于传统 RAG: 为 AI Agents 重新组织数据, 所有入库数据可以直接作为 LLM/VLM 的输入



思考: OpenViking 为什么是数据库而不是文件系统?



VikingMem-  
VLDB26-Final.pdf



上下文结构重组和文件分解, 是 OpenViking 在知识组织和检索方面的核心设计, 其优势是:

1. 让大的、多模态的文件变成有组织的小文件, 让 Agent 直接可读
2. 通过分层摘要和层次化组织, 让 LLM 不再需要阅读全部信息, 而是按需阅读
3. 小文件 (作为切片) 也让 Embedding 模型可以直接处理语义



# 记忆系统和知识检索系统的迭代

OpenViking 的 LLM Wiki 最佳实践：是将记忆和知识用 Wiki 的形态串联起来

## 3 — Frontmatter Schema

Every wiki page starts with flat YAML frontmatter. No nested objects. Obsidian's Properties UI doesn't support them.

Universal fields (every page):

```
---
type: <source|entity|concept|domain|comparison|question|overview|meta>
title: "Human-Readable Title"
created: 2026-04-07
updated: 2026-04-07
tags:
  - <domain-tag>
  - <type-tag>
status: <seed|developing|mature|evergreen>
related:
  - "[[Other Page]]"
sources:
  - "[[.raw/articles/source-file.md]]"
---
```

Type-specific additions:

source: source\_type, author, date\_published, url, confidence (high|medium|low), key\_claims (list)

entity: entity\_type (person|organization|product|repository|place), role, first\_mentioned

concept: complexity (basic|intermediate|advanced), domain, aliases (list)

comparison: subjects (list of wikilinks), dimensions (list), verdict (one line)

question: question (the original query), answer\_quality (draft|solid|definitive)

| 维度   | 传统 RAG              | LLM Wiki                        |
|------|---------------------|---------------------------------|
| 知识形态 | 切片 + 向量 (黑盒)        | Markdown 文件(白盒, 可 git diff)     |
| 是否人读 | 只供 LLM              | 同时供人和 LLM                       |
| 是否复利 | 否, 每次重检索            | 是, 编译产物会越攒越密                    |
| 错误纠正 | 改文档 / 重新嵌入          | 改 Markdown / 提交 PR              |
| 结构语义 | 段落级 chunk           | 文件即概念, wikilink 即关系             |
| 信任边界 | 难溯源                 | 每节都可挂 <S1> 源引用                  |
| 工具链  | 向量库 + Embedding API | Obsidian + Git + Markdown + LLM |
| 主要风险 | 召回低、上下文挤压           | 编译质量、写入冲突、噪声边                   |



# 记忆系统和知识检索系统的迭代

OpenViking 的 LLM Wiki 最佳实践：是将记忆和知识用 Wiki 的形态串联起来

## 3 — Frontmatter Schema

Every wiki page starts with flat YAML frontmatter. No nested objects. Obsidian's Properties UI doesn't support them.

Universal fields (every page):

```
---
type: <source|entity|concept|domain|comparison|question|overview|meta>
title: "Human-Readable Title"
created: 2026-04-07
updated: 2026-04-07
tags:
  - <domain-tag>
  - <type-tag>
status: <seed|developing|nature|evergreen>
related:
  - "[[Other Page]]"
sources:
  - "[[.raw/articles/source-file.md]]"
---
```

Type-specific additions:

source: source\_type, author, date\_published, url, confidence (high|medium|low), key\_claims (list)

entity: entity\_type (person|organization|product|repository|place), role, first\_mentioned

concept: complexity (basic|intermediate|advanced), domain, aliases (list)

comparison: subjects (list of wikilinks), dimensions (list), verdict (one line)

question: question (the original query), answer\_quality (draft|solid|definitive)

| 维度   | 传统 RAG              | LLM Wiki                        |
|------|---------------------|---------------------------------|
| 知识形态 | 切片 + 向量 (黑盒)        | Markdown 文件(白盒, 可 git diff)     |
| 是否人读 | 只供 LLM              | 同时供人和 LLM                       |
| 是否复利 | 否, 每次重检索            | 是, 编译产物会越攒越密                    |
| 错误纠正 | 改文档 / 重新嵌入          | 改 Markdown / 提交 PR              |
| 结构语义 | 段落级 chunk           | 文件即概念, wikilink 即关系             |
| 信任边界 | 难溯源                 | 每节都可挂 <S1> 源引用                  |
| 工具链  | 向量库 + Embedding API | Obsidian + Git + Markdown + LLM |
| 主要风险 | 召回低、上下文挤压           | 编译质量、写入冲突、噪声边                   |

OpenViking 决定将 LLM Wiki 抽象为一种格式, 专门用于建立上下文之间的关系。激活 LLM Wiki 的方法非常简单, 你可以在添加资源时, 通过 `--reason` 的参数描述添加文件的主观信号。

ov add-resource <https://arxiv.org/pdf/2605.29640> --reason “这是我们最近发表的论文”

得益于 OpenViking 通过文件系统范式进行上下文统一存储, 因此 LLM Wiki 可以在直接通过内在的链接 `viking://` 完成信息的统一引用



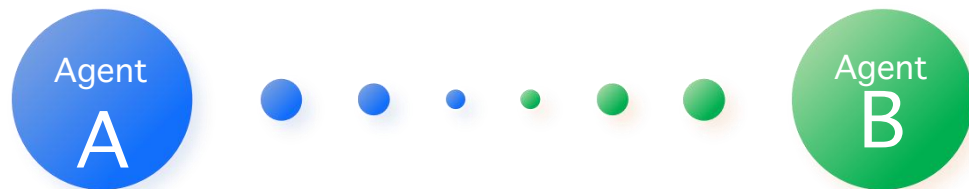
# 03 OpenViking 应用场景和评估

<https://openviking.ai/>

<https://github.com/volcengine/OpenViking>

# 多智能体协作

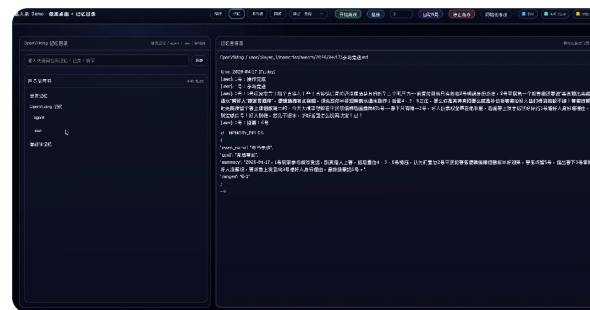
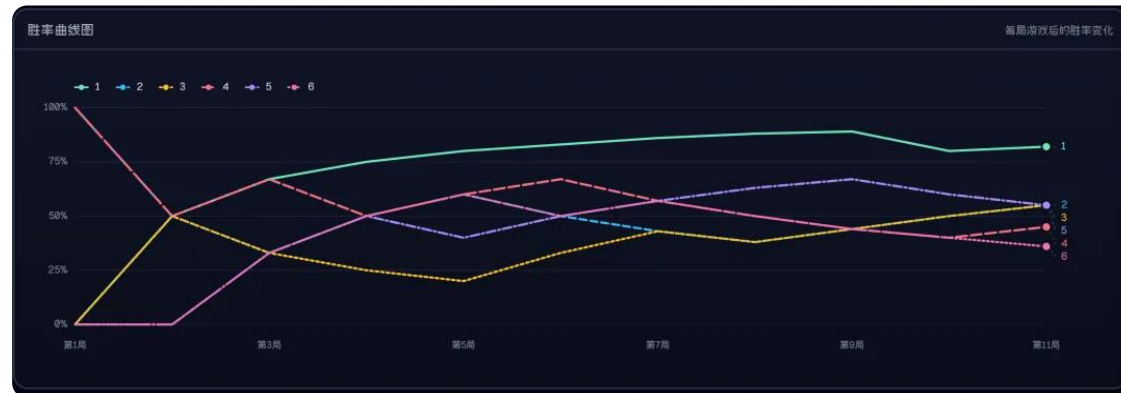
OpenViking 提供了上下文共享的新协作模式，区别于调用和对话通信



我已经开始了任务A，相关过程在 `viking://users/{owner_id}/agent_a` 目录下，请帮我调研 `viking://resources/project-A` 项目中关于 xxx 的实现，调研我们的实施方案一的可行性

好的，我已经查看过你的上下文，已经完成 project-A 的调研，我的结论是 ...

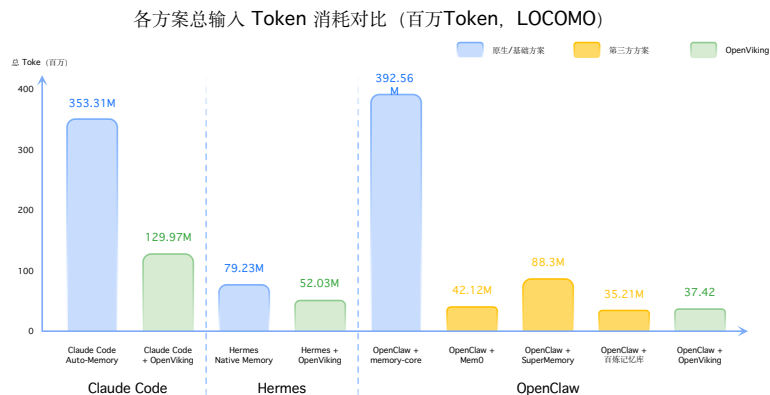
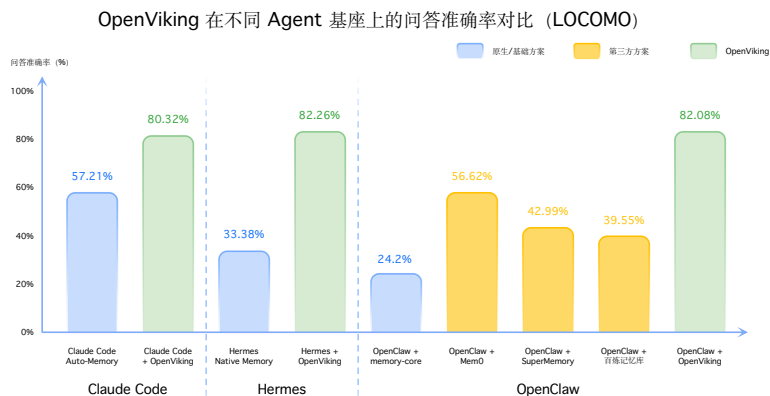
谢谢，我看到只有你能使用 xxx 工具，请帮我执行 zzz 任务



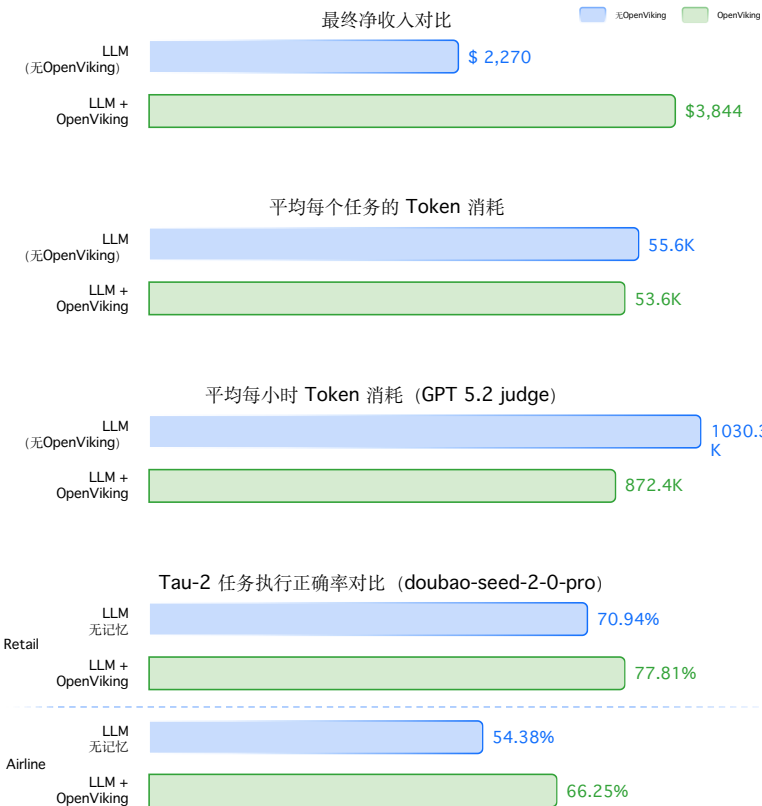
OpenViking 的记忆机制为多 Agent 协作提供了新的可能性，在一个基于《狼人杀》玩法的有趣实验中，有 OpenViking 支撑的智能体胜率显著超过无记忆的智能体

# 记忆能力

## 用户记忆效果 (LOCOMO)



## Agent 经验记忆效果



在记忆能力的评估中，我们主要关注：

1. 历史相关问题的正确率
2. 任务完成加速
3. Token 消耗减少的幅度

OpenViking 相对于不同 Agent 原生底座，可以大幅提高长对话和重复类长任务的完成率

其他数据集如 LongMemEval、Tau2 等也体现了类似的结论，OpenViking 记忆能力保持在记忆领域开源产品的第一梯队

# 知识检索和问答任务

与其他开源 RAG 框架比较 (HotpotQA)

| 方案                           | 检索范式           | Accuracy ↑ | 每 QAToken ↓ | 每 QA耗时 ↓ |
|------------------------------|----------------|------------|-------------|----------|
| Naive RAG                    | 向量检索           | 62.50%     | 1,290       | 0.11s    |
| HippoRAG 2                   | 向量 + 知识图谱      | 61.00%     | 726         | 20s      |
| LightRAG                     | 向量 + 知识图谱      | 89.00%     | 28,443      | 75s      |
| LangChain SQL (Agent)        | SQL 查询 (Agent) | 78.00%     | 4,776       | 132s     |
| OpenViking (top5召回)          | 向量检索           | 72.75%     | 3,154       | 0.22s    |
| OpenViking (top20召回)         | 向量检索           | 91.00%     | 12,533      | 0.23s    |
| Nanobot + OpenViking (Agent) | 向量检索 + Agent   | 87.00%     | 71,300      | 61.6s    |

在知识检索和问答能力的评估中，我们主要关注

1. 客观信息检索和问答的准确率

2. 任务完成加速

3. Token 消耗减少的幅度

OpenViking 相对于多个开源 RAG 框架

以及 Agent + 裸文件系统，都能在可控的成本下获取更好的精度

与裸文件系统比较

| 数据集              | 方案                   | Accuracy ↑ | 单词<br>QA Token ↓ | 平均<br>迭代轮次↓ |
|------------------|----------------------|------------|------------------|-------------|
| FinanceBench     | Nanobot 目录结构         | 0.822      | 208,836          | 13.5        |
|                  | Nanobot 扁平结构         | 0.670      | 206,784          | 16          |
|                  | OpenViking + Nanobot | 0.858      | 79,714           | 5.6         |
| QASPER           | Nanobot 目录结构         | 0.798      | 93,747           | 8.9         |
|                  | Nanobot 扁平结构         | 0.825      | 86,350           | 7.7         |
|                  | OpenViking + Nanobot | 0.847      | 59,504           | 5.5         |
| SyllabusQA       | Nanobot 目录结构         | 0.828      | 98,740           | 8.1         |
|                  | Nanobot 扁平结构         | 0.836      | 46,737           | 4.6         |
|                  | OpenViking + Nanobot | 0.862      | 46,708           | 4.8         |
| NaturalQuestions | Nanobot 目录结构         | 0.783      | 63,168           | 5.1         |
|                  | Nanobot 扁平结构         | 0.859      | 54,373           | 5.0         |
|                  | OpenViking + Nanobot | 0.923      | 43,118           | 4.2         |
| HotpotQA         | Nanobot 目录结构         | 0.870      | 71,300           | 6.3         |
|                  | Nanobot 扁平结构         | 0.888      | 58,950           | 5.3         |
|                  | OpenViking + Nanobot | 0.893      | 73,003           | 5.1         |

# 工程和代码任务

OpenViking 的上下文集成能力，可以把代码、文档、架构说明、历史讨论汇聚在一起，在软件工程中帮助分析复杂问题

Viking 团队在软件开发任务中需要处理上百个代码仓库和大量的文档库，传统 RAG 框架无法处理如此复杂的上下文场景。在 OpenViking 的帮助下，我们搭建起了“团队级的上下文基础设施”，智能体帮助解决各类客户问题

随着大模型长上下文工作能力持续提升，对今天的智能体而言，我们完全可以将需求PRD、客户反馈记录、线上代码、接口文档、部署架构、甚至群聊讨论都集成到同一个上下文数据库（OpenViking）中

打破团队边界，提供更多的上下文，可以获得非常明显的效率加成

|      | 较好         | 一般         | 较差         | 输入 Token 成本 | 输入Token成本中位数 |
|------|------------|------------|------------|-------------|--------------|
| 对照组  | 4/10 (40%) | 3/10 (30%) | 3/10 (30%) | 625,183     | 486,128      |
| 实验组一 | 8/10 (80%) | 1/10 (10%) | 1/10 (10%) | 323,294     | 292,722      |
| 实验组二 | 9/10 (90%) | 1/10 (10%) | 0/10 (0%)  | 216,331     | 213,863      |

关于多仓库问题检索和回答的测评

[https://mp.weixin.qq.com/s/qNDa34R00LR\\_Nveo\\_6wEhw](https://mp.weixin.qq.com/s/qNDa34R00LR_Nveo_6wEhw)

15:46  
@OpenViking小助手 Hermes接入之后，保存到OpenViking是自动触发的还是被动触发呢？  
OpenViking小助手  
1 条话题回复

12:57  
@Viking诊断助手 VikingDB 有没有直接计算文本相似度得分的 API，有无text + text → similarity\_score 的一步式接口  
Viking诊断助手  
Viking诊断助手  
6 条话题回复

OpenViking小助手 机器人 | 基于 | 点赞 | 评论 | 分享 | ...

根据Hermes官方文档，我现在可以回答用户的问题了。文档明确说明了Hermes内存提供程序的工作机制。根据Hermes官方文档，Hermes接入OpenViking后，保存到OpenViking是自动触发的。

具体工作机制如下：

自动触发的保存机制

当OpenViking作为内存提供程序激活时，Hermes会自动：

1. 每次响应后同步对话轮次 - 在每次AI响应后自动将会话内容同步到OpenViking
2. 会话结束时提取记忆 - 在会话结束时自动提取记忆（支持profile、preferences、entities、events、

Viking诊断助手 智能体 | 基于 | 点赞 | 评论 | 分享 | ...

是的，VikingDB 有可以直接计算文本相似度得分的 API！

答案

VikingDB 提供了 rerank 接口，可以直接计算两段文本之间的相似度得分。这个接口位于知识库 SDK 中，而不是向量库 SDK 中。

主要功能

- 接口可以接受查询文本（query）和待比较文本（content）对
- 返回与输入一一对应的相似度得分数组
- 分数越高表示两段文本越相关

支持的模型

# 性能、成本和效率

OpenViking 的目标是支持大规模上下文集成，我们最近几个月在性能、成本和效率方面的投入有了一些明显的进展



解决 **Scaling Out** 的核心是允许分布式部署架构，虽然 OpenViking 的早期架构约束是允许单机部署，但是在规模化场景（如企业AI工具、团队研发协同、智能客服等），几十甚至几百 **QPS** 的请求是单机无法承载的

1. OpenViking Service 作为无状态服务层，支持多实例部署模式，分散处理请求流量
2. 通过锁和队列，优化内部数据处理管线，让数据写入接口改为异步任务化处理，增强数据一致性承诺
3. 依托公有云 VikingDB 超高索引性能，将查询、统计、过滤等重计算任务外包到 VikingDB 完成，降低无状态层负荷
4. 支持多种分布式文件系统，并引入数据缓存层，大幅改善文件系统层容量限制
5. 完善日志和遥测接口，提供更加完善的性能分析和优化观测点位



# 性能、成本和效率

OpenViking 希望用户能用最低成本试用起来，优化 Token 成本、降低部署门槛

以火山方舟提供的豆包大模型（2.0）为例，早期记忆抽取算法，Pro/Lite 模型存在较大 Gap，但在 0.3 版本中，通过优化记忆抽取机制，我们大幅提升了 Lite 模型的测试表现，使其接近 Pro 模型

Lite 模型的价格只有 Pro 的 1/5，与 embedding 模型相当

| Eval result    | model        | 单sample最长耗时 | Emb token         | Vim token           | Cache token | Reasoning token | Out put          |
|----------------|--------------|-------------|-------------------|---------------------|-------------|-----------------|------------------|
| 80.78%         | code-2.0     |             | 1,360,407         | 4,303,687           | 1,931,856   | 0               | 647,664          |
| 81.69%         | 2.0-pro-0215 | 47min21.34s | 1,845,934         | 5,116,580           | 1,925,408   | 0               | 1,001,118        |
| 78.31% (↓3.38) | lite-0428    | 15min17.04s | 787,127 (↓57.36%) | 3,355,737 (↓34.41%) | 1,665,352   | 0               | 411905 (↓58.86%) |
| 73.44% (↓8.25) | mini-0428    | 8min9.77s   | 628,815 (↓65.94%) | 3,326,374 (↓34.99%) | 1,915,064   | 0               | 427014 (↓57.35%) |

# 极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

## AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

## QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

## AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

# AiCon

全球人工智能开发与应用大会

深圳站

## 探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

### 专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



### 专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



### 专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



### 专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



### 专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



### 专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO





# 04 OpenViking 的愿景和展望

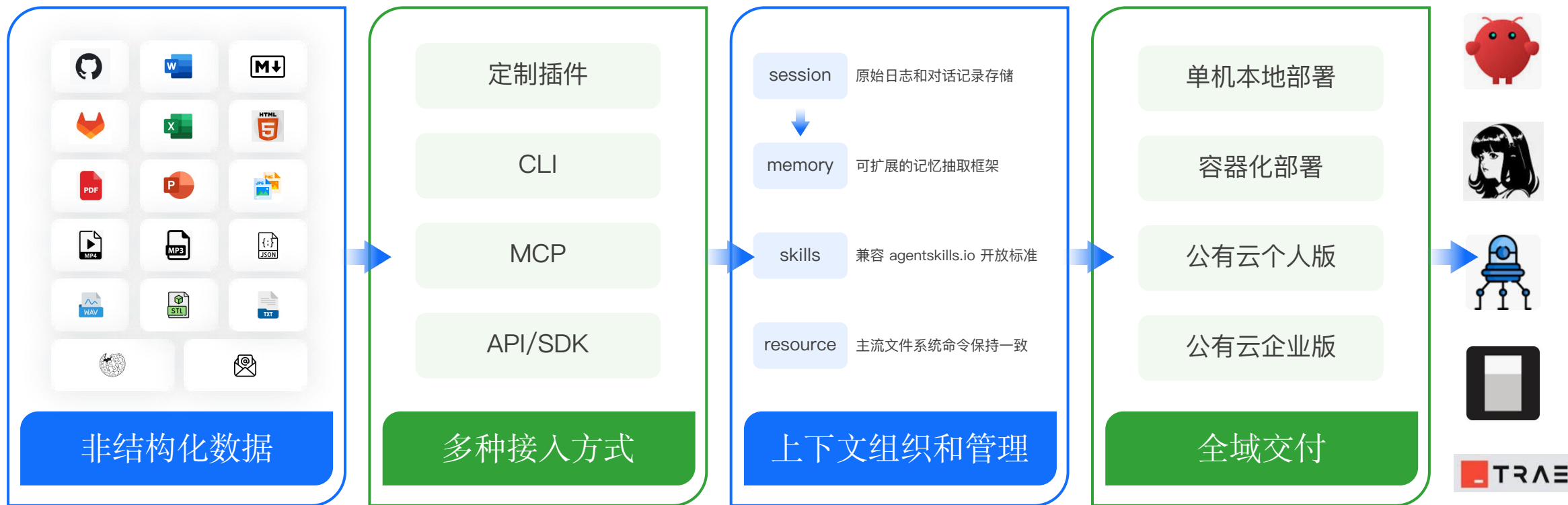
<https://openviking.ai/>

<https://github.com/volcengine/OpenViking>

# ■ 服务于 AI 原生团队

OpenViking 正在成为 AI 原生团队的首选：

一站式搭建智能体上下文基建、统一管理各类 Agent 的记忆，系统解决上下文工程问题



# 参与构建安全可靠的智能体软件栈

OpenViking 在文件系统的基础上叠加了经过打磨的租户权限管理和数据安全能力，允许放心交付重要数据



## OpenViking 安全设计包括：

1. 文件存储加密
2. RBAC权限控制、清晰的数据归属定义
3. 多层级防越权保护
4. 对接API-KEY、SSO等多种鉴权协议
5. KMS密钥管理系统、强制密钥认证
6. HTTPS通信保护
7. 运行态容器级隔离

我们在公有云上默认开启所有数据安全特性,最高级别保护用户数据安全

# THANKS

探索 AI 应用边界

Explore the limits of AI applications

## AiCon

全球人工智能开发与应用大会



欢迎飞书扫码加入开发者社区群